Mastering OpenMPI

A Comprehensive Study Guide

From Shared Memory to Distributed HPC Clusters

Collective Operations · Non-Blocking I/O · Derived Datatypes

4th Year AI • ENSIA • February 2026

Introduction

Modern scientific challenges — climate modelling, genomics, fluid dynamics, machine learning at scale — demand far more compute than any single machine can provide. This study guide covers two lecture modules on OpenMPI, the industry-standard open-source MPI implementation. Module 1 introduces the fundamentals: why distributed computing is necessary, the SPMD execution model, and point-to-point communication. Module 2 covers collective operations, non-blocking I/O, and advanced derived datatypes.

After working through this document you will be able to: explain the architectural difference between OpenMP and OpenMPI; write correct MPI programs using both point-to-point and collective communication; overlap computation with network transfers; and safely transmit non-contiguous and heterogeneous data structures.

Part 1 — Foundations of Distributed Computing with OpenMPI

1. Why Distributed Computing?

For decades, CPU performance grew by raising clock speeds and adding transistors to a single chip. This approach eventually hit a hard physical ceiling called the memory wall: as chips grow more powerful, heat dissipation and memory bandwidth become bottlenecks that no amount of chip design can overcome.

The answer is horizontal scaling — connect many ordinary machines over a fast network and let them cooperate. The theoretical limit on computation becomes the size of the cluster, not the physics of a single die.

Grand-challenge problems make this concrete. A global climate model tracks the atmosphere, oceans, ice sheets, and land surface at fine resolution, generating petabytes of data no single node could handle. Whole-genome sequencing pipelines compare billions of base-pair fragments — a task that is embarrassingly parallel and ideal for large clusters.

2. The Paradigm Shift: OpenMP vs. OpenMPI

Two major parallel programming models dominate HPC. Understanding their difference is prerequisite to writing any MPI code.

OpenMP — Shared Memory

Scope: Single node only

Memory: All threads share the same RAM

Pros: Easier to program; implicit data sharing

Cons: Hard limit — bounded by motherboard resources

OpenMPI — Distributed Memory

Scope: Across multiple nodes (machines)

Memory: Each process has its own private RAM

Pros: Scales to thousands of machines

Cons: Requires explicit message passing

> **!** **Key Insight**
> Because MPI processes cannot share memory, the only way one process can use data that lives in another process's memory is to send it over the network as a message. This is the central programming challenge of MPI.

3. A Brief History of Message Passing

Before the MPI standard existed, each supercomputer vendor provided its own parallel library (PVM, PARMACS, etc.). Code written for one machine was virtually non-portable — a serious problem for reproducible science.

- Pre-1992 — Fragmented, vendor-specific libraries. Non-portable code.
- 1994 (MPI-1.0) — First unified standard. Portable API for point-to-point and collective communication.
- 1997 (MPI-2.0) — Parallel I/O, dynamic process management, one-sided (remote memory access) communication.
- 2012 (MPI-3.0) — Non-blocking collective operations.
- Present (OpenMPI) — Open-source, production-grade implementation powering today's Exascale supercomputers.

4. The SPMD Execution Model

MPI programs follow the SPMD paradigm — Single Program, Multiple Data. You compile one executable. That same binary is launched simultaneously on every processor or node. Each instance uses its unique numerical ID (Rank) to determine which data to process and which code branches to execute.

Think of it like handing every worker the same instruction sheet with a different section number printed on it. The instructions say "process your section" — everyone follows the same instructions, but on different data.

> **!** **Important Pattern**
> The `if (rank == 0)` pattern appears constantly in MPI code. It allows one specific process (the root) to perform I/O, distribute work, or print results, while all others handle computation.

5. Structure of an MPI Program

Every MPI program in C/C++ must follow a strict scaffolding:

C / MPI Code

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    // Step 1 — Initialize the MPI runtime.
    // MUST be the first MPI call.
    MPI_Init(&argc, &argv);
```

```
    // Step 2 — All parallel work and MPI communication goes here.

    // Step 3 — Shut down MPI cleanly.
    // No MPI calls allowed after this.
    MPI_Finalize();
    return 0;
}
```

`MPI_Init` sets up the runtime, connects processes, and prepares the communication infrastructure. Forgetting it is undefined behaviour. Forgetting **`MPI_Finalize`** may leave network resources in an inconsistent state.

5.1 Communicators, Rank, and Size

- **Communicator (MPI_COMM_WORLD) —** A named group of processes that can exchange messages. MPI_COMM_WORLD is the default communicator containing every launched process.
- **Size —** Total number of processes in a communicator. Retrieved with MPI_Comm_size().
- **Rank —** The unique integer ID (0 to Size−1) of each process. Retrieved with MPI_Comm_rank().

C / MPI Code

```
int world_size, world_rank;

// How many processes are running in total?
MPI_Comm_size(MPI_COMM_WORLD, &world_size);

// Which process am I?
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
```

By dividing N total work items into world_size chunks and having rank world_rank process chunk number world_rank, you achieve load-balanced parallelism automatically.

5.2 Hello World in MPI

C / MPI Code

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(NULL, NULL);
    int world_size, world_rank;
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    printf("Hello from rank %d out of %d\n",
           world_rank, world_size);

    MPI_Finalize();
    return 0;
}
```

Running with mpirun -np 4 ./hello launches 4 processes. The output order is not guaranteed — you might see rank 2 before rank 0. This non-determinism is your first encounter with asynchronous distributed execution. The demo (hostname.c) additionally calls MPI_Get_processor_name() to print which physical machine each process is running on, making the distributed nature of the execution visible.

6. Point-to-Point Communication

Point-to-point communication is the most fundamental MPI mechanism: one process sends data directly to one other specific process. The sender must specify the data, the destination rank, and a message tag. The receiver must specify where to put the data, which sender it expects, and the matching tag.

6.1 MPI_Send and MPI_Recv

C / MPI Code

```
// Sending data:
int MPI_Send(
    void*        data,          // Pointer to send buffer
    int          count,         // Number of elements to send
    MPI_Datatype datatype,      // e.g. MPI_INT, MPI_DOUBLE
    int          destination,   // Rank of receiver
    int          tag,           // Message label (any non-negative integer)
    MPI_Comm     communicator   // e.g. MPI_COMM_WORLD
);

// Receiving data:
int MPI_Recv(
    void*        data,          // Pointer to receive buffer
    int          count,         // Max elements expected
    MPI_Datatype datatype,
    int          source,        // Expected sender rank (or MPI_ANY_SOURCE)
    int          tag,           // Must match sender tag (or MPI_ANY_TAG)
    MPI_Comm     communicator,
    MPI_Status*  status         // Info about the received message
);
```

! **Important: Blocking Calls**

Both MPI_Send and MPI_Recv are blocking. MPI_Send will not return until the buffer is safe to reuse. MPI_Recv will not return until the entire message has arrived. This means each process waits for the other — a key performance consideration addressed in Part 2.

6.2 Deadlock Warning

A deadlock occurs when all processes are waiting for a message that will never arrive because everyone is waiting to receive before anyone sends. Classic solution for a ring topology: even-ranked processes send first then receive; odd-ranked processes receive first then send. This breaks the circular dependency.

6.3 MPI Datatypes

MPI needs to know exactly how many bytes to transmit — it cannot infer this from a C type alone because byte representations may differ between machine architectures in a heterogeneous cluster. Use the portable MPI type constants:

C Type	MPI Type Constant
`short`	`MPI_SHORT`
`int`	`MPI_INT`
`long`	`MPI_LONG`
`float`	`MPI_FLOAT`
`double`	`MPI_DOUBLE`
`char`	`MPI_CHAR`

For complex data structures (C structs with mixed types), MPI provides derived datatypes — covered in detail in Section 9.

Part 2 — Collective Operations, Non-Blocking I/O, and Derived Datatypes

7. Collective Communications

Point-to-point communication requires routing data manually between every pair of processes. For common parallel patterns — distributing an array, aggregating partial results — this is tedious and error-prone. Collective operations provide high-level, hardware-optimised functions that involve all processes simultaneously.

- **Simplicity —** A single function call replaces a complex loop of sends and receives.
- **Efficiency —** MPI implementations use hardware-aware algorithms (binary tree broadcasts, pipelined reduces) far faster than linear point-to-point.
- **Readability —** The algorithmic intent is immediately clear.

7.1 Golden Rules of Collectives

> **!** **Critical Rule — Universal Participation**
> Every single process in the communicator MUST call the collective function. If even one process skips it (e.g., due to an if-statement), the entire program deadlocks forever. No exceptions.

- **No tags —** Collectives are matched by their execution order, not by a tag integer.
- **Blocking by default —** A collective call does not return until the process's participation is complete and its buffers are safe to reuse.

7.2 Synchronization: MPI_Barrier

MPI_Barrier creates a synchronisation point. No process can pass it until every process in the communicator has arrived. Use it to: (1) accurately time a parallel region by ensuring all processes start together, and (2) coordinate file I/O (ensure all writes are done before the root reads).

C / MPI Code

```
int MPI_Barrier(MPI_Comm comm);
```

7.3 Broadcasting Data: MPI_Bcast

Purpose: one root process holds data (e.g., a configuration value) that every other process needs a copy of. MPI_Bcast sends the same data from root to all, using an efficient tree-based algorithm (O(log N) steps vs. O(N) for a naive loop).

C / MPI Code

```
int MPI_Bcast(
    void*        buffer,  // Root: data to send; Others: receive buffer
    int          count,   // Number of elements
    MPI_Datatype type,
    int          root,    // Rank of the sending process
    MPI_Comm     comm
);
```

7.4 Distributing Work: MPI_Scatter

Purpose: the root divides a large array into equal chunks and distributes one chunk to each process (including itself).

C / MPI Code

```
int MPI_Scatter(
    void*        sendbuf,    // Root only: full array
    int          sendcount,  // Elements per process — NOT total size!
    MPI_Datatype sendtype,
    void*        recvbuf,    // All: local receive buffer
    int          recvcount,  // Elements each process receives
    MPI_Datatype recvtype,
    int          root,
    MPI_Comm     comm
);
```

! **Common Mistake**
sendcount is the number of elements sent to EACH process, not the total array size. For 100 elements across 4 processes, sendcount = 25.

7.5 Collecting Results: MPI_Gather

MPI_Gather is the exact inverse of Scatter. Every process sends its local data chunk to the root, which assembles all chunks into a single array in rank order. The Scatter → parallel compute → Gather pattern is MPI's equivalent of map-reduce.

C / MPI Code

```
int MPI_Gather(
    void*        sendbuf,   // All: local chunk
```

```
    int          sendcount, // Elements each process sends
    MPI_Datatype sendtype,
    void*        recvbuf,   // Root only: output array
    int          recvcount, // Elements received from EACH process
    MPI_Datatype recvtype,
    int          root,
    MPI_Comm     comm
);
```

7.6 Global Aggregation: MPI_Reduce

MPI_Reduce collects one value (or array) from every process, applies an associative operation element-wise, and places the final result on the root. MPI_Allreduce delivers the result to every process instead of just the root.

C / MPI Code

```
int MPI_Reduce(
    void*        sendbuf,  // Local contribution
    void*        recvbuf,  // Final result (valid only on root after call)
    int          count,
    MPI_Datatype type,
    MPI_Op       op,       // MPI_SUM, MPI_MAX, MPI_MIN, MPI_PROD ...
    int          root,
    MPI_Comm     comm
);
```

!

Key Insight

The speedup demo (speedup.c) uses MPI_Reduce with MPI_SUM to aggregate 100 million sin(i)×cos(i) values computed in parallel. Each process computes a partial sum; MPI_Reduce collapses them into the final answer on rank 0, which then prints the speedup factor and efficiency percentage.

8. Non-Blocking Communication

Standard MPI_Send and MPI_Recv are blocking: the calling process stalls until the network transfer is complete. In HPC, an idle CPU is a wasted resource. Non-blocking variants let you hide network latency behind useful computation.

8.1 MPI_Isend and MPI_Irecv

C / MPI Code

```
// Non-blocking send — returns immediately; transfer runs in background
int MPI_Isend(void *buf, int count, MPI_Datatype type,
              int dest, int tag, MPI_Comm comm,
              MPI_Request *request);  // handle to track progress

// Non-blocking receive — returns immediately; data fills buffer later
int MPI_Irecv(void *buf, int count, MPI_Datatype type,
              int source, int tag, MPI_Comm comm,
              MPI_Request *request);
```

! **Critical Rule**
After MPI_Isend, do NOT modify the send buffer until transfer is confirmed complete. After MPI_Irecv, do NOT read the receive buffer until confirmed complete. Violating this silently corrupts data.

8.2 MPI_Wait and MPI_Test

C / MPI Code

```
// BLOCKING wait — suspend until this specific transfer finishes
int MPI_Wait(MPI_Request *request, MPI_Status *status);

// NON-BLOCKING poll — set flag=1 if done, flag=0 if still in progress
int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status);
```

The recommended overlapping pattern:

- Step 1 — Call MPI_Isend / MPI_Irecv to start the transfer.
- Step 2 — Perform independent CPU computations that do not touch the buffers.
- Step 3 — Call MPI_Wait to block only now, when you actually need the data.
- Step 4 — Proceed with code that depends on the transferred data.

Demo demo_overlap.c demonstrates that switching from blocking MPI_Send to MPI_Isend with computation in between produces a dramatic drop in total execution time because network latency is hidden behind useful work.

9. Derived Datatypes for Non-Contiguous Memory

MPI works perfectly for contiguous data (a 1D array of ints). But many HPC data structures are non-contiguous: a column of a C 2D matrix (stored row-major) has gaps between elements; a C struct has compiler-inserted padding bytes between fields. The naive solution — manually copying into a temporary buffer ("packing"), sending, then unpacking — wastes CPU time and doubles memory. Derived Datatypes let you describe the layout to MPI once so it can read bytes directly from their original locations.

9.1 MPI_Type_vector — Strided Data (e.g., Matrix Columns)

C / MPI Code

```
int MPI_Type_vector(
    int          count,       // Number of blocks to collect
    int          blocklength, // Consecutive elements per block
    int          stride,      // Gap between block starts (in elements)
    MPI_Datatype old,         // Base type (e.g. MPI_DOUBLE)
    MPI_Datatype *newtype     // Output: the new custom type
);
```

```
MPI_Type_commit(&newtype);   // REQUIRED before any use
// ... use newtype in MPI_Send / MPI_Recv ...
MPI_Type_free(&newtype);     // REQUIRED when done — prevents memory leaks
```

Example: sending column 2 of a 5×5 matrix stored as a flat 1D array of doubles. Set count=5 (five rows), blocklength=1 (one element per row), stride=5 (each row is 5 doubles wide). MPI selects elements at offsets 2, 7, 12, 17, 22 — exactly the column — without any manual copying.

9.2 MPI_Type_create_struct — Heterogeneous C Structs

To transmit an entire C struct as one MPI message, you must describe its layout using byte offsets computed at runtime with the standard C macro offsetof(). This correctly accounts for compiler-inserted padding.

C / MPI Code

```
#include <stddef.h>   // for offsetof()

struct Particle {
    int    id;         // 4 bytes
    double velocity;   // 8 bytes (compiler may insert padding before this)
    char   active;     // 1 byte
};

int          blocklengths[3] = {1, 1, 1};
```

```
MPI_Datatype types[3]        = {MPI_INT, MPI_DOUBLE, MPI_CHAR};
MPI_Aint     offsets[3];
offsets[0] = offsetof(struct Particle, id);
offsets[1] = offsetof(struct Particle, velocity);
offsets[2] = offsetof(struct Particle, active);

MPI_Datatype particle_type;
MPI_Type_create_struct(3, blocklengths, offsets, types, &particle_type);
MPI_Type_commit(&particle_type);

// Send an entire array of Particles in one call:
MPI_Send(particles, count, particle_type, dest, tag, MPI_COMM_WORLD);

MPI_Type_free(&particle_type);
```

!

Why offsetof()?

The C standard permits compilers to insert invisible padding bytes between struct fields for alignment. These bytes must be included in the MPI layout description. offsetof() returns the exact byte position of each field as seen by the current compiler, making the description portable across platforms.

Summary and Key Takeaways

10. Summary Table

Function / Concept	Summary
`SPMD Model`	One binary; each process uses Rank to select its data and code branch.
`MPI_Send / MPI_Recv`	Blocking point-to-point. Simple but CPU sits idle waiting for the network.
`MPI_Barrier`	Synchronisation point. All processes wait until everyone has arrived.
`MPI_Bcast`	Root sends the same data to every process. O(log N) tree algorithm.
`MPI_Scatter`	Root splits an array into equal chunks; one chunk per process. sendcount = chunk size, NOT total.
`MPI_Gather`	Each process sends its chunk to root, assembled in rank order. Inverse of Scatter.
`MPI_Reduce`	One value per process aggregated by an operation (SUM, MAX …) on root. MPI_Allreduce delivers result to all.
`MPI_Isend / MPI_Irecv`	Non-blocking. Return immediately; transfer runs in background. Enable computation-communication overlap.
`MPI_Wait / MPI_Test`	Confirm completion of non-blocking requests before accessing buffers.
`MPI_Type_vector`	Describes strided layouts (matrix columns). Avoids manual packing.
`MPI_Type_create_struct`	Describes heterogeneous C structs. Uses offsetof() to handle compiler padding.

10.1 Communication Pattern Decision Guide

- One process to one other process? → MPI_Send / MPI_Recv (blocking) or MPI_Isend / MPI_Irecv (if you have parallel work to hide the latency).
- Every process needs the same data from root? → MPI_Bcast.
- Divide an array evenly across all workers? → MPI_Scatter.
- Collect processed chunks from all workers to root? → MPI_Gather.

- Combine a value from every process into a global aggregate? → MPI_Reduce or MPI_Allreduce.
- Data is non-contiguous (struct fields, matrix columns)? → Derived datatype: MPI_Type_vector or MPI_Type_create_struct.

!

Final Exam Reminder

The most tested topics are: (1) SPMD model and rank/size usage, (2) deadlock avoidance in point-to-point, (3) the Golden Rule of Collectives (ALL processes must participate), (4) the non-blocking overlap pattern (Isend → compute → Wait), and (5) the difference between MPI_Scatter's sendcount (per-process chunk size) and the full array size.